
UNIDAD 07

Arrays unidimensionales

Programación en Computación · Ingeniería Electromecánica — 2.º año · UTN FR Reconquista
Longhi Pablo · Talijancic Iván

Qué vamos a ver

01 El **problema** que resuelven los vectores

02 Declarar, inicializar y acceder por **índice**

03 **Recorrer** con `for` y con `while`

04 `.slice()` : la trampa de las **referencias**

05 Los **cinco algoritmos** fundamentales

06 Probar y depurar en la **consola** de Node

07 **Taller:** ahora ustedes

Requisitos: unidad 05 (bucles) y unidad 06 (funciones). Todo lo de hoy se escribe dentro de funciones.

PARTE 1

| El problema

Tres mediciones de tensión

src/app.js

```
let measurement1 = 219.4  
let measurement2 = 221.8  
let measurement3 = 218.2
```

Funciona perfecto. Tres valores, tres variables.

Ahora las de todo el mes

src/app.js

```
let measurement1 = 219.4  
let measurement2 = 221.8  
let measurement3 = 218.2  
// ... 27 líneas más ...  
let measurement31 = 220.1
```

Inviabile. Y el problema real todavía no apareció.

PREGUNTA

¿Y si el usuario decide
cuántas mediciones son?

No podés declarar variables que todavía no sabés cuántas son.

Un nombre, muchos valores

src/app.js

```
let measurements = [219.4, 221.8, 218.2]
```

MEDICIONES

219.4	221.8	218.2
0	1	2

Todos los elementos comparten el **nombre**. Los distingue el **índice**.

PARTE 2

| Declarar y acceder

Tres formas de declarar

Vacío

```
let vector = []
```

Cuando no sabés la dimensión todavía.

Con valores

```
let vector = [1, 2, 3]
```

Cuando los datos son conocidos.

Con dimensión

```
let vector = new Array(5)
```

5 posiciones, todas sin inicializar.

En esta cátedra los vectores se inicializan **siempre con un bucle**. No usamos `fill()` ni `push()`.

Leer y modificar

src/app.js

```
let names = ['Juan', 'Ana', 'Luis']  
  
console.log(names[0])  
console.log(names[2])  
  
names[1] = 'María'  
console.log(names)
```

SALIDA

```
Juan  
Luis  
[ 'Juan', 'María', 'Luis' ]
```

.length: cuántos elementos tiene

src/app.js

```
let vector = ['Juan', 'Ana', 'Luis']  
console.log(vector.length) // 3
```

CONCEPTO

VALOR

Cantidad de elementos

`vector.length`

Índice del primero

0

Índice del último

`vector.length - 1`

3 ELEMENTOS • ÚLTIMO ÍNDICE 2

'Juan'	'Ana'	'Luis'
0	1	2

PREGUNTA

¿Qué imprime `names[3]`?

```
let names = ['Juan', 'Ana', 'Luis']  
console.log(names[3])
```

undefined, y sin avisar

src/app.js

```
let names = ['Juan', 'Ana', 'Luis']  
  
console.log(names[3])  
console.log(names[3] + 1)
```

SALIDA

```
undefined  
NaN
```

ERROR FRECUENTE

JavaScript **no tira error** al salirse de rango. El programa sigue con basura adentro y el error explota mucho más adelante, lejos de donde lo causaste. **El control del rango es tu responsabilidad.**

PARTE 3

| Recorrer el vector

Con **for**

src/app.js

```
const printVector = (vector) => {  
  for (let i = 0; i < vector.length; i++) {  
    console.log(`${i}: ${vector[i]}`)  
  }  
}  
  
printVector([10, 20, 30])
```

SALIDA

```
0: 10  
1: 20  
2: 30
```

Es la forma más común: sabés exactamente cuántas iteraciones necesitás, tantas como elementos tenga el vector.

< 0 <=

Para un vector de **5 elementos**:

CONDICIÓN	ÚLTIMO I	ACCEDE A	
<code>i < vector.length</code>	4	índices 0, 1, 2, 3, 4	✓
<code>i <= vector.length</code>	5	<code>vector[5]</code> → <code>undefined</code>	✗

ERROR FRECUENTE

Cinco elementos, **último índice 4**. Es el error número uno al empezar con vectores.

Con `while`

src/app.js

```
let i = 0

while (i < vector.length) {
  console.log(vector[i])
  i++
}
```

- 01 **Inicializar** el contador antes del bucle
- 02 **Condición** de corte
- 03 **Incrementar** dentro del bucle

ERROR FRECUENTE

Si te olvidás el `i++`, la condición nunca se hace falsa: **bucle infinito**. Se corta con `Ctrl + C` en la terminal.

¿Cuál uso?

for

Recorrer el vector completo.
Sabés cuántas iteraciones son.

while

Cortar antes de llegar al final.
La cantidad depende de una condición.

`for` por defecto. `while` cuando necesitás cortar.

PARTE 4

| La trampa de `.slice()`

Un número se pasa por copia

src/app.js

```
const doubleAll = (n) => {  
  n = n * 2  
  return n  
}  
  
let x = 5  
const y = doubleAll(x)
```

SALIDA

```
x = 5      ← intacto  
y = 10
```

La función recibe una **copia** del valor. El original no se toca.

Un vector, no

src/app.js

```
const doubleAllBad = (vector) => {  
  for (let i = 0; i < vector.length; i++) {  
    vector[i] = vector[i] * 2  
  }  
  return vector  
}  
  
const original = [1, 2, 3]  
const doubled = doubleAllBad(original)
```

PREGUNTA

¿Cuánto vale `original` ahora?

```
const original = [1, 2, 3]
const doubled = doubleAllBad(original)

console.log(original)
```

Se arruinó el original

SALIDA

```
Original: [ 2, 4, 6 ]    ← se arruinó  
Duplicado: [ 2, 4, 6 ]
```

Número

Se pasa por **copia**. La función trabaja sobre un valor propio.

Vector

Se pasa por **referencia**. Apunta al mismo lugar en memoria.

La solución

× ASÍ NO

```
const doubleAll = (vector) => {  
  for (...) {  
    vector[i] = vector[i] * 2  
  }  
  return vector  
}
```

✓ ASÍ SÍ

```
const doubleAll = (vector) => {  
  const result = vector.slice()  
  for (...) {  
    result[i] = result[i] * 2  
  }  
  return result  
}
```

Usá `.slice()` cuando una función **recibe** un vector que no debe modificar, y cuando **retorna** uno.

PARTE 5

| Los cinco algoritmos

Acumular

src/app.js

```
const sumAll = (vector) => {  
  let total = 0  
  
  for (let i = 0; i < vector.length; i++) {  
    total += vector[i]  
  }  
  
  return total  
}
```

TRAZA CON [5, 10, 15]

I	V[I]	SUMA
—	—	0
0	5	5
1	10	15
2	15	30

El acumulador arranca en 0 y se declara **fuera** del bucle. Adentro, se reinicia en cada vuelta.

Contar según una condición

src/app.js

```
const countEven = (vector) => {  
  let count = 0  
  
  for (let i = 0; i < vector.length; i++) {  
    if (vector[i] % 2 === 0) {  
      count++  
    }  
  }  
  
  return count  
}
```

Igual que acumular, pero el contador sube **de a uno** y sólo cuando se cumple el `if`.

Máximo y su posición

src/app.js

```
const findMax = (vector) => {  
  let max = vector[0] // el PRIMERO, no 0  
  let position = 0  
  
  for (let i = 1; i < vector.length; i++) { // arranca en 1  
    if (vector[i] > max) {  
      max = vector[i]  
      position = i  
    }  
  }  
  
  return [max, position].slice() // dos datos → un vector  
}
```

PREGUNTA

¿Por qué no `let max = 0`?

```
findMax([-5, -12, -3, -40])
```

Con temperaturas bajo cero

VECTOR DE TRABAJO

-5	-12	-3	-40
0	1	2	3

INICIALIZANDO EN

```
let max = 0
```

```
let max = vector[0]
```

RESULTADO

0 — un valor que no está en el vector ❌

-3 ✅

ERROR FRECUENTE

En los parciales esto se corrige como **error grave**: denota no entender qué representa el acumulador.

Búsqueda lineal

src/app.js

```
const linearSearch = (vector, wanted) => {  
  let position = -1  
  let i = 0  
  
  while (i < vector.length && position === -1) { // corta al encontrarlo  
    if (vector[i] === wanted) { position = i }  
    i++  
  }  
  
  return position // -1 si no está  
}
```

-1 es un índice **imposible**: así distinguís «no está» de «está en la posición 0».

Vector aleatorio

src/app.js

```
const rndInt = (min, max) =>
  Math.floor(Math.random() * (max - min + 1)) + min

const getRandomVector = (size, min, max) => {
  const vector = new Array(size)

  for (let i = 0; i < size; i++) {
    vector[i] = rndInt(min, max)
  }

  return vector.slice()
}
```

`rndInt()` ya la escribimos en la unidad 06. Se reusa de acá en adelante.

PARTE 6

| Probar en la consola

Probar sin crear un archivo

terminal

```
$ node
> const temps = [62, 68, 71, 83]
undefined
> temps.length
4
> temps[temps.length - 1]
83
> temps[4]
undefined
```

Escribís `node` en la terminal, sin archivo, y cada línea se ejecuta **en el momento**.

El `undefined` después de un `const` **no es un error**: es lo que devuelve una declaración. El de `temps[4]` sí te está avisando algo.

Para salir: `.exit` o `Ctrl + D` dos veces.

Depurar una función

terminal

```
> const findMax = (vector) => {  
|   let max = 0  
|   for (let i = 0; i < vector.length; i++) {  
|     if (vector[i] > max) { max = vector[i] }  
|   }  
|   return max  
| }  
undefined  
> findMax([62, 68, 91, 74])  
91  
> findMax([-5, -12, -3])  
0
```

Pegás la función y la llamás con los casos **difíciles**. Sin `npm run dev`, sin cargar datos.

ERROR FRECUENTE

`0` no está en el vector. Con todos negativos, el error de `let max = 0` aparece en el segundo intento.

Lo que conviene saber

TECLA O COMANDO	QUÉ HACE
↑ / ↓	Recorre lo que ya escribiste: corregís una línea sin volver a tipearla
Tab	Autocompleta: <code>Math.</code> + Tab lista <code>floor</code> , <code>random</code> , <code>round</code> ...
-	El último resultado
Ctrl + C	Cancela la línea que estás escribiendo
<code>.save prueba.js</code>	Guarda en un archivo todo lo que probaste
<code>.help</code>	Lista todos los comandos

En el taller que sigue: escribí la función en `src/app.js`, y **antes de usarla en el programa**, probala acá.

PARTE 7

| Ahora ustedes

Los datos del taller

Un motor tiene un sensor en el rodamiento. Se registra la temperatura **una vez por hora**. Por encima de **80 °C** el rodamiento entra en alarma.

62	68	71	83	79	85	74	91
0	1	2	3	4	5	6	7

src/app.js

```
const ALARM_LIMIT = 80
const temperatures = [62, 68, 71, 83, 79, 85, 74, 91]
```

A1 • Pasar toda la serie a Fahrenheit

Devolvé un **vector nuevo** con las temperaturas en °F. El vector original **no se toca**.

src/app.js

```
const celsiusToFahrenheit = (vector) => // ...  
  
console.log(celsiusToFahrenheit(temperatures))  
console.log(temperatures)
```

SALIDA

```
[ 143.6, 154.4, 159.8, 181.4, 174.2, 185, 165.2, 195.8 ]  
[ 62, 68, 71, 83, 79, 85, 74, 91 ]
```

La fórmula es la del TP de la unidad 05: $^{\circ}\text{C} * 9 / 5 + 32$.

A2 • Cuántas en alarma

`countAbove(vector, limit)` devuelve **cuántos** elementos superan el límite.

src/app.js

```
const countAbove = (vector, limit) => // ...  
  
console.log(countAbove(temperatures, ALARM_LIMIT))
```

SALIDA

```
3
```

Contador de la unidad 05, ahora **adentro de una función** y sobre un vector.

B1 • La primera en alarma

`firstAbove(vector, limit)` devuelve el **índice** del primer elemento que supera el límite, o `-1` si no hay ninguno.

src/app.js

```
const firstAbove = (vector, limit) => // ...  
  
console.log(firstAbove(temperatures, ALARM_LIMIT))  
console.log(firstAbove([62, 68, 71], ALARM_LIMIT))
```

SALIDA

```
3  
-1
```

Acordate de `isPrime` : en cuanto lo encontrás, `return i` — y el bucle **se corta solo**.

B2 • ★ Sólo las que están en alarma

`aboveLimit(vector, limit)` devuelve un **vector nuevo** con los elementos que superan el límite. Nada más.

src/app.js

```
const aboveLimit = (vector, limit) => // ...  
  
console.log(aboveLimit(temperatures, ALARM_LIMIT))
```

SALIDA

```
[ 83, 85, 91 ]
```

ERROR FRECUENTE

`new Array(...)` necesita saber **el tamaño**. ¿Y cuántos elementos va a tener este vector? No lo sabés hasta recorrerlo. Y `push()` no se usa.

Lo que acaban de construir

01 `aboveLimit` usa `countAbove` para saber de qué tamaño crear el vector

02 Dos funciones devuelven un **vector nuevo**, y ninguna toca el que recibió

03 `firstAbove` devuelve `-1`: un índice imposible que significa «no está»

Un vector no se filtra de una pasada: **primero se cuenta, después se copia.**

HASTA QUE EN OTRA MATERIA LES DEN LAS HERRAMIENTAS QUE LO HACEN SOLAS.

Los errores que vamos a cometer

ERROR	SÍNTOMA
<code>i <= vector.length</code>	<code>undefined</code> o <code>NaN</code> al final
Acumulador dentro del bucle	Sólo queda el último elemento
<code>let max = 0</code>	Falla con valores negativos
Olvidar <code>i++</code> en un <code>while</code>	El programa se cuelga
Olvidar <code>parseInt()</code>	La suma concatena: <code>'53'</code>
Modificar el vector recibido	Se corrompen los datos de entrada

Reglas de la cátedra

Podés usar

`for` · `while` · `do-while` · `if` · `switch` · acceso por índice
· `.length` · `.slice()`

No podés usar

`map` · `filter` · `reduce` · `forEach` · `find` · `sort` · `flat` ·
`indexOf` · `splice` · `fill()` · `push()` · retornar objetos

Primero dominás la lógica del recorrido.

LAS HERRAMIENTAS QUE LA RESUELVEN POR VOS VIENEN DESPUÉS

TRABAJO PRÁCTICO

Análisis de mediciones de tensión de línea

7 problemas · todo modularizado en funciones · `tp.md`

Apunte completo de la unidad: `apunte.md`